Hertie School

Natural Language Processing

Session 8: Neural Models II
Reading in order · MLP · CNNs · RNNs

GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz · l.ruiz@hertie-school.org

Announcements

Research Note one-pager:

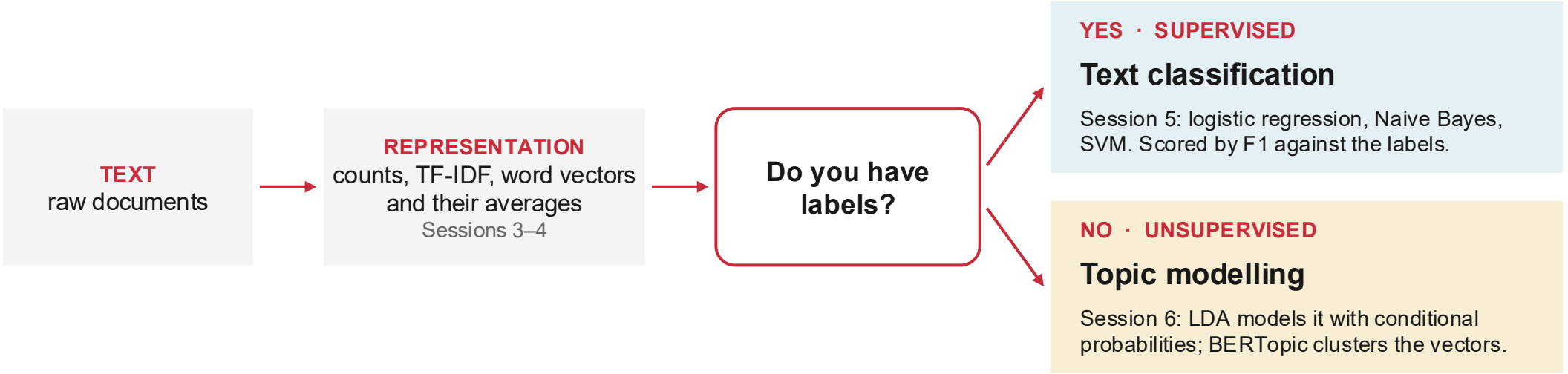
- Due this week (session 8).

Quiz 3: next week (session 9):

- Covers sessions 6, 7 and 8

So far: the first divide is whether you have labels

Sessions 3–6 in one picture: code the text as features, then ask one question.



SESSION 7: NEURAL NETWORKS JOIN THE YES BRANCH

Same question, same F1, but a more flexible f(x): it learns non-linear combinations of the features you coded.

TODAY'S TWIST

Language modelling makes its own labels from raw text, so the no branch can borrow the yes branch's tools (slide 18).

Still the first question for your research note: do you have labels, and what exactly do they record?

What neural networks add: four big ideas

Neural networks keep the pipeline. They change who builds the features, and how much of the text the model reads.

1

Non-linearity

keyword: interactions

Hidden layers learn that “not” and “bad” matter only together.

Session 7

2

Representation

keyword: meaning

Word2vec learned word vectors before the task; a network learns its own, for the task.

Sessions 4, 7

3

Context

keyword: order

A window (CNN), then a running memory (RNN). Every word at once: Session 9.

today

4

Language modelling

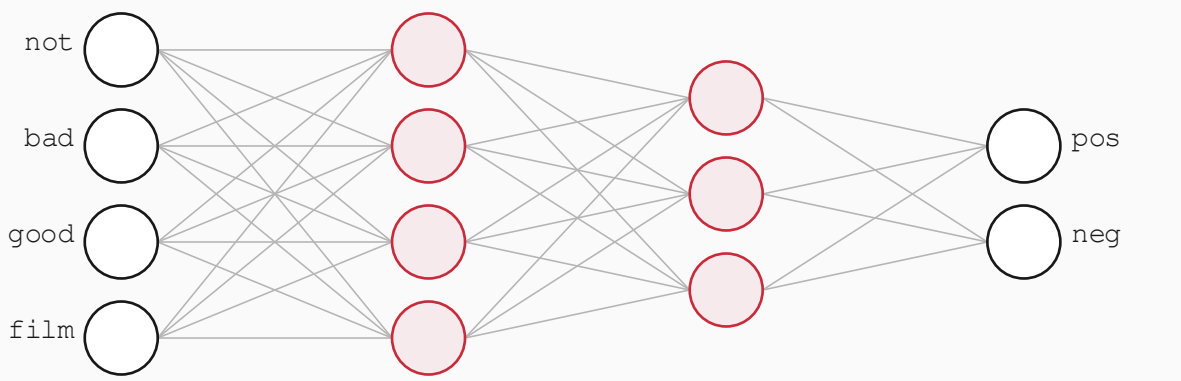
keyword: next word

Predict the next word: raw text becomes labelled data, with no annotator.

today → Session 9

Ideas 3 and 4 drive each other: more context gives better next-word guesses, which give a richer semantic space. Together they blur the first divide: pre-train on unlabelled text, then fine-tune on your labels.

Recap: an MLP stacks hidden layers, then classifies



x: the features you coded

$\mathbf{h_1 = f(W_1 x + b_1)}$

new coordinates

$\mathbf{h_2 = f(W_2 h_1 + b_2)}$

coordinates of coordinates

$\mathbf{\hat{y} = softmax(W_3 h_2 + b_3)}$

a logistic regression on h_2

THE TRAINING LOOP

1. **Forward pass:** compute ŷ for a batch of documents.
2. **Loss:** compare ŷ with the labels.
3. **Backward pass:** backpropagation sends the blame from the loss to every weight, layer by layer.
4. **Update:** $w \leftarrow w - \eta \cdot \partial L/\partial w$, for every weight.

WHAT AN MLP READS

One fixed-size vector, so a text of any length is first summarised: counts, or the average of its word vectors. A strong baseline, blind to order.

Each hidden layer re-describes the one before it. Logistic regression finds a line in the space you give it; an MLP first learns the space.

Average the word vectors, or keep the sequence

SESSIONS 4–6 · AVERAGE: ONE VECTOR PER DOCUMENT

the	0.0	0.0	0.1
heat	0.1	0.4	0.0
was	0.0	0.0	0.1
not	0.9	0.1	0.0
bad	0.0	0.9	0.0

mean →

the document

0.20	**0.28**	**0.04**

- **Meaning survives:** similar words, similar averages.
- **Order is gone, by construction:** “the heat was not bad” and “bad was not the heat” give the same mean. Counts lose it the same way.

TODAY · SEQUENCE: ONE ROW PER TOKEN, IN ORDER

t		d_1	d_2	d_3
1	the	0.0	0.0	0.1
2	heat	0.1	0.4	0.0
3	was	0.0	0.0	0.1
4	not	0.9	0.1	0.0
5	bad	0.0	0.9	0.0

THE SENTENCE MATRIX
5 tokens × d dims; row t is token t. Shuffle the words and the matrix changes.

Where the rows come from: an embedding layer, a lookup table that can start from Word2vec (Session 4), frozen or fine-tuned, or be learned from scratch for the task.

Averaging keeps what the words mean but forgets their order; the sentence matrix keeps both. An MLP needs one fixed-size vector, so we usually feed it the average; CNNs and RNNs read the matrix directly.

What CNNs aim to solve

WHAT AN AVERAGE OR A BAG CANNOT DO	THE CNN'S ANSWER
1 Order inside a phrase "not bad" and "bad, not" give the same vector.	→ A filter reads the k words of its window in order: "not bad" ≠ "bad not".
2 Phrases nobody listed a hand-made "not bad" column says nothing about "not terrible".	→ Filters read embeddings, so similar words score alike: "not terrible" ≈ "not bad".
3 Anywhere in the text the same phrase must count at word 3 or at word 300.	→ The same filter slides over every position; max-pooling keeps its best match.

WHAT IT LEAVES OPEN

Anything longer than the window: "not, as many feared, a bad film".

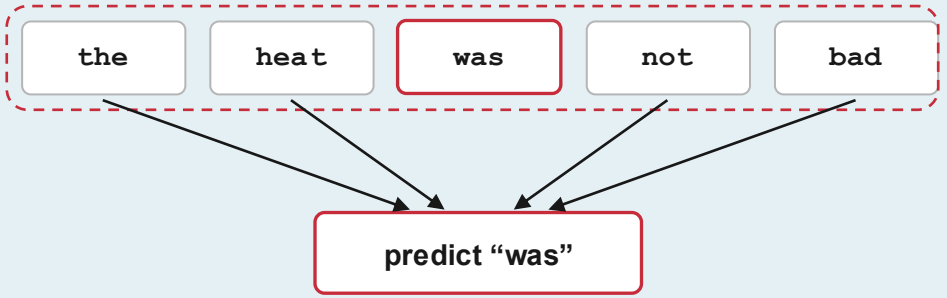
Where and in what order patterns occur, once pooled.

→ **a memory that follows the whole sequence: RNNs, after the break.**

CNNs turn hand-made n-grams into learned, shared phrase detectors. They read order locally, never across the whole text.

Same sliding window, two different jobs

WORD2VEC · SESSION 4



Window: ± 2 words around a centre word.

Job: predict the centre word from its neighbours: the distributional hypothesis.

Leaves behind: one fixed vector per word. Order inside the window is ignored.

CNN FILTER · TODAY



Window: k = 2 words, in order.

Job: score a word pattern that helps predict your label.

Leaves behind: a score at every position. “not bad” ≠ “bad not”.

Word2vec used a window to learn what words mean; a CNN uses one to find which word sequences matter for your task.
The two combine: a CNN can read Word2vec’s vectors.

CNNs for text: how a 1D filter reads two words at a time

SENTENCE: 5 WORDS × 3 DIMS

	d_1	d_2	d_3
the	0.0	0.0	0.1
heat	0.1	0.4	0.0
was	0.0	0.0	0.1
not	**0.9**	**0.1**	**0.0**
bad	**0.0**	**0.9**	**0.0**

toy embeddings; real ones have 300 dims

FILTER: 2 WORDS × 3 DIMS

row 1: word 1 is a negation
row 2: word 2 is negative

1	**0**	**0**
0	**1**	**0**

bias b = −1

ONE POSITION: THE WINDOW “not bad”

1 · multiply cell by cell, then add everything

```
not: 0.9·1 + 0.1·0 + 0.0·0  =  0.9
bad: 0.0·0 + 0.9·1 + 0.0·0  =  0.9
```

2 · add the bias: 0.9 + 0.9 − 1 = 0.8

3 · ReLU(0.8) = 0.8: the pattern is there

The same as one neuron on the two vectors joined.

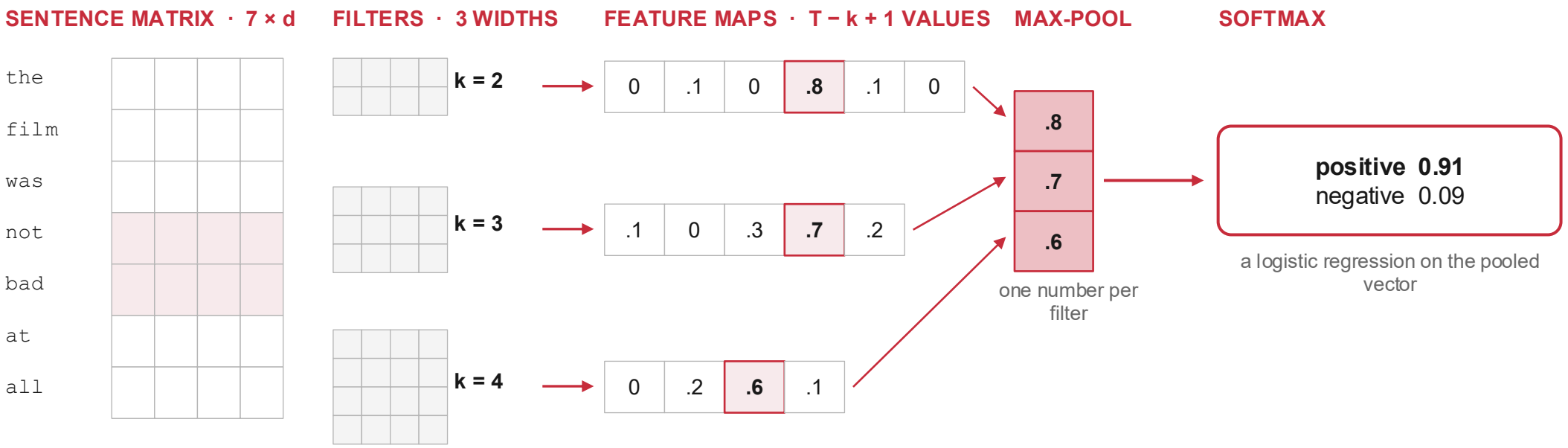
SLIDE TO EVERY POSITION, THEN POOL

the heat → 0 | heat was → 0 | was not → 0 | **not bad → 0.8** → **max-pool → 0.8**

Like a bigram, but learned and graded. Session 5’s “not bad” column was 0 or 1 for one exact pair. This filter scores any pair that looks like negation + negative word: “not terrible” scores 0.7, “was bad” scores 0.

A filter is one neuron for k-word windows, reused at every position. 100 filters are 100 learned phrase detectors, pooled into 100 numbers for the softmax.

A CNN text classifier, end to end



WHY MAX-POOL OVER TIME

Keeps whether each pattern occurred, anywhere, and turns a sentence of any length into a fixed-length vector.

WHY IT IS FAST

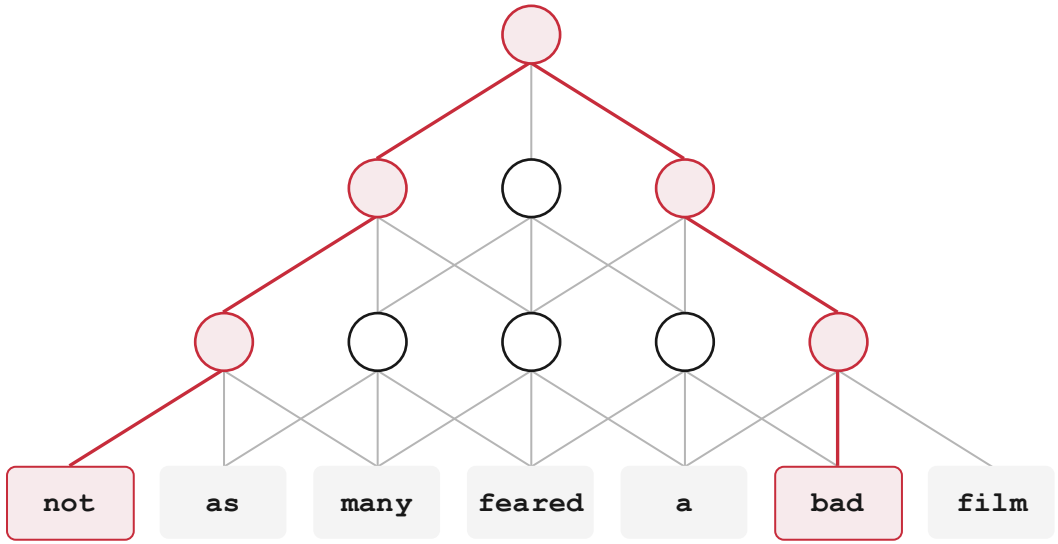
Every window is computed independently, all at once: ideal for a GPU. No word waits for the one before it.

IN THE PAPER

Kim (2014): widths 3, 4 and 5, 100 filters each, so 300 numbers reach the softmax. Strong on sentiment and question type.

A CNN classifier is a bank of learned phrase detectors, of several lengths, followed by a logistic regression on what they found.

Where CNNs stop: the window is the limit



k = 3 at every layer. Red: the first unit whose view holds both “not” and “bad”, three layers up.

ONE LAYER: TOO NARROW

No window of 3 holds both “not” and “bad”: the filter sees “not as many” and “feared a bad”, never the negation of “bad”.

STACKING WIDENS THE VIEW

3, 5, 7 words. A long article needs many layers, and every layer needs more data.

POOLING FORGETS WHERE

“good at first, terrible by the end” and “terrible at first, good by the end” light up the same filters.

CNNs are excellent at local order and poor at long-range order. To carry “not” across a detour, a model needs a memory: after the break.

Discuss: phrase detectors for your corpus

1 Name three short phrases a CNN's filters would probably learn for your research note's task. Which signal in your texts lives more than five words apart?

2 Max-pooling keeps whether a pattern occurred, not where. Write one sentence from your corpus where the position or order of two phrases changes the label.

3 A CNN and Session 5's logistic regression with bigrams reach the same F1 on your data. Which one do you report, and what would change your mind?

3 min in pairs → 3 min discussion.

Session Break

When we return: networks that read one word at a time and remember: RNNs.

What RNNs aim to solve

WHAT A FIXED WINDOW CANNOT DO	THE RNN'S ANSWER
1 Dependencies of any length "not … bad" can be 2 words apart or 12; a window has one fixed size.	→ A memory h_t is carried from word to word, so anything read so far can still matter.
2 A whole sequence, in order we want one representation of the text so far, for any length, that changes when words move.	→ h_t is updated at every word with the same weights: it stands for the prefix w_1 … w_t.
3 Writing, not only reading language models and translation produce text one word at a time.	→ Read h_t at each step, predict the next word, feed it back in, repeat.

WHAT IT LEAVES OPEN

Memory fades with distance (slide 19).

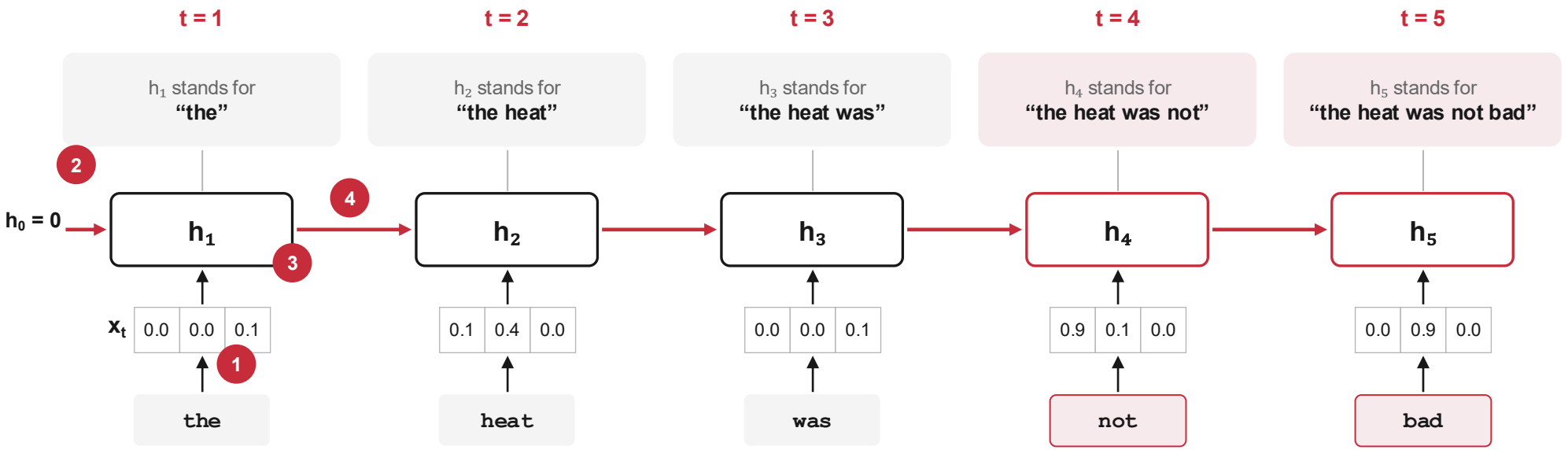
One step after another: slow to train.

One vector must hold a whole sentence (slide 20).

→ **attention: every word looks at every other directly. Session 9.**

RNNs replace the fixed window with a running memory. Their three weaknesses, on the right, are exactly what transformers were built to fix.

An RNN, step by step: read a word, update the memory, repeat



1 Look up

the word's vector x_t in the embedding table. Static: "bad" always gets the same three numbers.

2 Bring the memory

h_{t-1} from the previous step; at t = 1 it is all zeros.

3 Combine and squash

$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$: one layer, the same weights at every step.

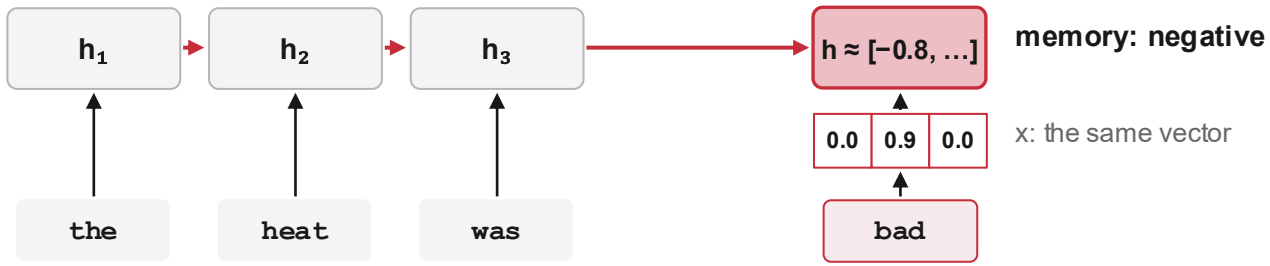
4 Pass it on

h_t now stands for every word read so far. Go to the next word.

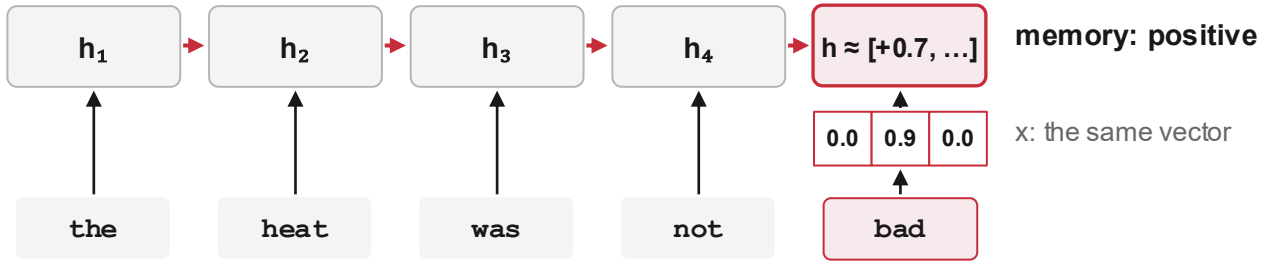
One small network, applied word after word, passing its memory to itself (Elman, 1990). After the last word, h_5 is a representation of the whole sentence, built in order.

Static vectors in, dynamic memory out

A · "the heat was bad"



B · "the heat was not bad"



STATIC · x_t

Looked up in the embedding table, like Word2vec (Session 4): "bad" is always [0.0, 0.9, 0.0].

DYNAMIC · h_t

Computed from everything read so far: the same word leaves a different memory in a different context. A contextual representation, of the prefix only.

IN SESSION 9

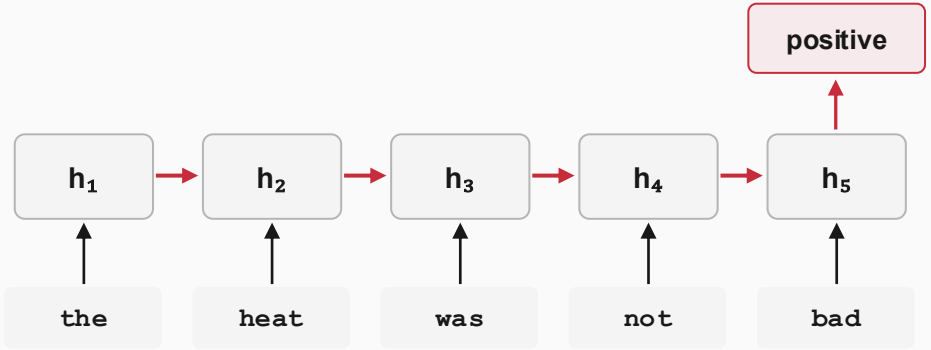
Transformers make every word's vector depend on the whole sentence, both sides, in one step: contextual embeddings proper.

Yes, an RNN gives each position a dynamic representation, but one that only looks back, and is built one word at a time.

One RNN, two ways to read it

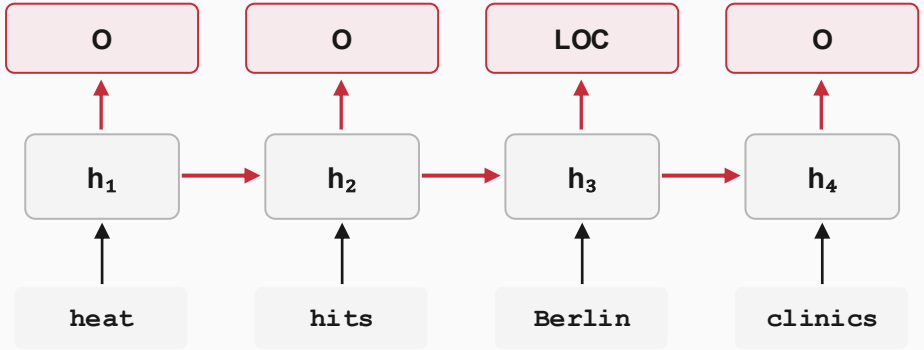
The recurrence is always the same. The task decides which "memories" h_t or representations go to a softmax.

MANY-TO-ONE · ONE LABEL PER TEXT



Only the last memory h_5, which has seen every word, goes to the softmax. Use: sentiment, stance, the topic of a post.

MANY-TO-MANY · ONE LABEL PER WORD

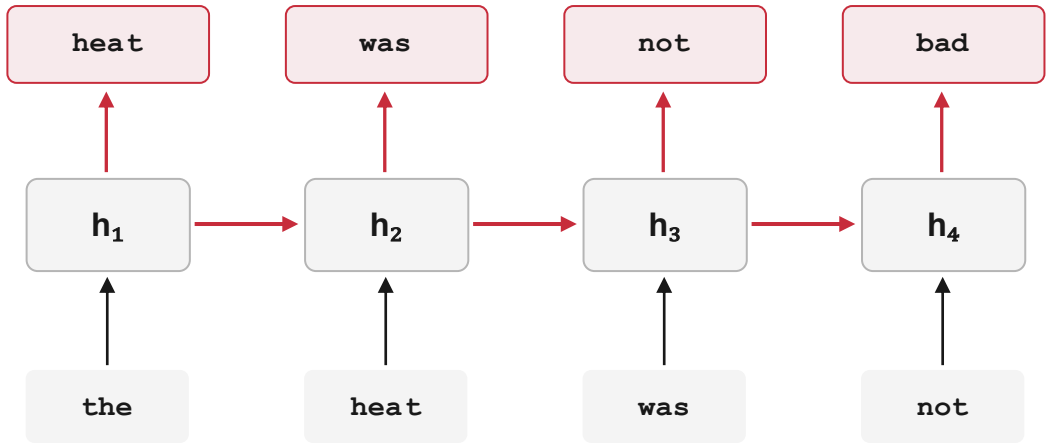


Every h_t gets its own softmax: one prediction per word, using everything read so far. Use: named entities, parts of speech, and next-word prediction.

Next: language modelling is many-to-many, where each word's label is simply the next word. And reading (many-to-one) followed by writing (a language model) is the encoder–decoder of slide 20.

Language modelling: the text labels itself

TARGETS = THE SAME TEXT, SHIFTED BY ONE WORD



AT THE LAST STEP: p(next word | the heat was not)



WHERE THE LABELS COME FROM

Every position in every sentence is a training example: 1 million sentences of 20 words give 20 million, with no annotator.

WHY CONTEXT HELPS

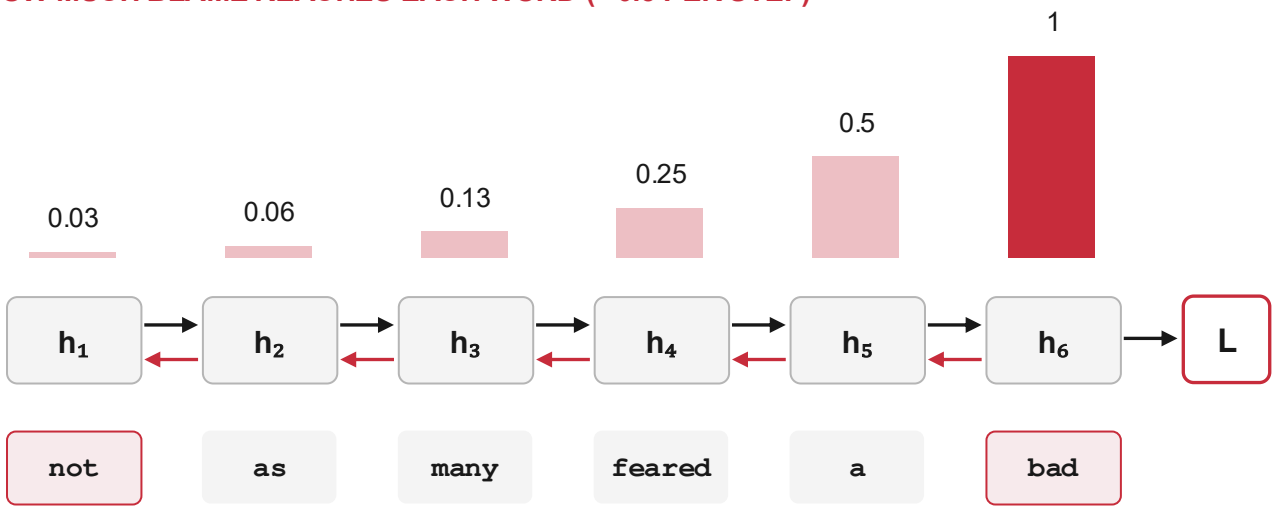
An n-gram model sees the last n − 1 words; an RNN conditions on the whole prefix, $p(w_t \mid w_1, \ldots, w_{t-1})$.

WHAT IT LEARNS ON THE WAY

To guess the next word, the hidden state must encode syntax, meaning and some facts: a semantic space as a by-product.

Training an RNN: blame fades as it travels back

HOW MUCH BLAME REACHES EACH WORD (× 0.5 PER STEP)



black: the forward pass · red: the backward pass, multiplied by about the same factor at every step

THE ARITHMETIC

factor per step	10 words	50 words
0.9	0.35	0.005
1.0	1	1
1.1	2.6	117

below 1: vanishes · above 1: explodes

Backpropagation through time: unroll the RNN into one deep network, one layer per word, and run Session 7's backpropagation. The same W_h sits in every layer, so word 1's blame is multiplied by it once per step.

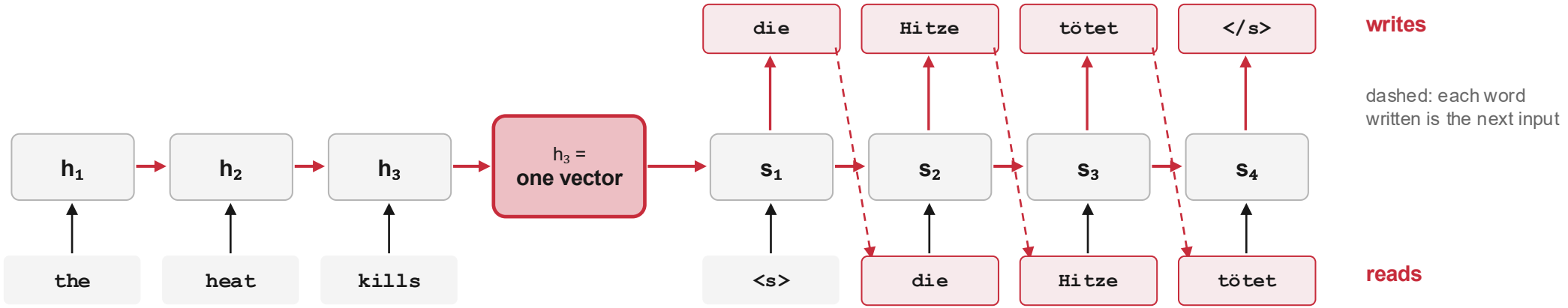
1
$$L = \sum_{t=1}^{T} L^{(t)}$$

2
$$\frac{\partial L^{(t)}}{\partial \boldsymbol{W}_{hh}} = \frac{\partial L^{(t)}}{\partial \boldsymbol{o}^{(t)}} \times \frac{\partial \boldsymbol{o}^{(t)}}{\partial \boldsymbol{h}^{(t)}} \times \left(\sum_{k=1}^{t} \frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} \times \frac{\partial \boldsymbol{h}^{(k)}}{\partial \boldsymbol{W}_{hh}} \right)$$

3
$$\frac{\partial \boldsymbol{h}^{(t)}}{\partial \boldsymbol{h}^{(k)}} = \prod_{i=k+1}^{t} \frac{\partial \boldsymbol{h}^{(i)}}{\partial \boldsymbol{h}^{(i-1)}}$$

A simple RNN can carry a memory, but cannot learn what to keep for long: the blame never reaches the early words (Bengio, Simard & Frasconi, 1994).

Read, then write: the encoder–decoder



ENCODER
the many-to-one reader (slide 17) without the softmax: its last memory is the sentence vector.

DECODER
a language model (slide 18) that starts from that vector instead of $h_0 = 0$, and feeds back each word it writes.

THE BOTTLENECK

One vector must hold the whole input; in long sentences the start is lost (Sutskever, Vinyals & Le, 2014).

THE FIX

Attention: let each output word look back at every encoder state, not only h_3 (Bahdanau, Cho & Bengio, 2015).

THE NEXT STEP

Keep attention, drop the recurrence: the transformer (Vaswani et al., 2017). Session 9.

RNNs gave us memory and next-word prediction; their fading memory, their slowness and this bottleneck are why transformers exist.

MLP, CNN, RNN: what each reads, and what it costs

	READS ORDER	HOW FAR IT SEES	PARALLEL?	USE IT FOR	WATCH OUT FOR
MLP Session 7	no: an average or a bag	the whole text, unordered	yes	quick baselines on small labelled sets	“not bad” = “bad, not”
CNN today	inside each window	k words per layer; more with depth	yes: all windows at once	classification by key phrases	pooling forgets where
RNN today	in sequence	in principle all; in practice a few words	no: word after word	language models; the idea of memory	fading memory; one-vector bottleneck

Next: the transformer reads every word against every other, all at once, with no bottleneck. Session 9.

Each row reads more of the order, and pays for it: CNNs with depth, RNNs with time. Pick the simplest one that reads enough for your question, and report it against Session 5’s logistic regression.

Discuss: memory, order and labels

1 Read like an RNN: “The minister, who had promised for years that the clinics would stay open, closed them.” What must the memory hold, and for how many words? With blame halving at every step, how much reaches “minister” from “closed”?

2 Your research note: is the task many-to-one, many-to-many, or unlabelled? Which row of the last table would you try first, and what result would make you switch?

3 A language model is trained only to predict the next word in news articles. Why might its hidden states help classify climate-health stories it was never labelled for?

3 min in pairs → 3 min discussion.

Where we are in the NLP pipeline



Data acquisition	Text extraction, normalization	Representation / features	Basic models	Neural models	Evaluation	Final evaluation + XAI	Orchestration
Not covered	Session 2	**Sessions 3–4**	Sessions 5–6	SESSIONS 7–9	Throughout	Session 10	Sessions 11-12

MLP CNNs RNNs · next: attention and transformers (Session 9)

Today: networks that read text in order. Next: networks that read every word at once.

References and links

READINGS

Raschka, Liu & Mirjalili (2022). Machine Learning with PyTorch and Scikit-Learn, ch. 15: RNNs.
course reading

Tunstall, von Werra & Wolf (2022). Natural Language Processing with Transformers, ch. 1.
course reading

CNNS AND RNNS

Kim (2014). Convolutional Neural Networks for Sentence Classification.
arxiv.org/abs/1408.5882

Elman (1990). Finding Structure in Time. Cognitive Science.
doi.org/10.1207/s15516709cog1402_1

Bengio, Simard & Frasconi (1994). Learning Long-Term Dependencies with Gradient Descent Is Difficult.
doi.org/10.1109/72.279181

Hochreiter & Schmidhuber (1997). Long Short-Term Memory.
doi.org/10.1162/neco.1997.9.8.1735

Cho et al. (2014). Learning Phrase Representations using RNN Encoder–Decoder (GRU).
arxiv.org/abs/1406.1078

LANGUAGE MODELS

Bengio, Ducharme, Vincent & Jauvin (2003). A Neural Probabilistic Language Model. JMLR.
jmlr.org/papers/v3/bengio03a.html

Mikolov et al. (2010). Recurrent Neural Network Based Language Model. Interspeech.
isca-archive.org/interspeech_2010/mikolov10_interspeech.html

Howard & Ruder (2018). Universal Language Model Fine-tuning for Text Classification (ULMFiT).
arxiv.org/abs/1801.06146

ENCODER–DECODER AND ATTENTION

Sutskever, Vinyals & Le (2014). Sequence to Sequence Learning with Neural Networks.
arxiv.org/abs/1409.3215

Bahdanau, Cho & Bengio (2015). Neural Machine Translation by Jointly Learning to Align and Translate.
arxiv.org/abs/1409.0473

Vaswani et al. (2017). Attention Is All You Need.
arxiv.org/abs/1706.03762